

Object Oriented Programming with Java 1 (2026)

Assignment 4

Niels-Christian Borbjerg
nielsb@setur.fo

May 1, 2026

1 Problem description

In this assignment you are to design and implement a puzzle board game similar to *Ricochet Robots*¹. This should be a console game that is based on principles of object oriented programming, enabling easy extendability of new features, rules or game elements.

The game takes place on a board with $r \times c$ cells — this means r rows and c columns — where a number of robots can be moved around. In figure 1 we see one example configuration of how the game board might be structured. Here AA (2,3), BB (1,1) and CC (4,7) are robots and gg (3,6) is the goal. Furthermore we have an outer wall, i.e. the board is entirely enclosed by walls — every valid board should have this property — and an inner wall.

Each robot starts on its own cell on the board, and the goal of the game is to move one robot to the goal using the least amount of moves. The particular challenge in this arises from the rule, that when a robot moves in one direction, it will keep moving in that direction until it hits a wall, another robot or perhaps some other

```
+--+--+--+--+--+--+--+
| BB                |
+  +  +  +--+--+  +  +
|      AA |        |
+  +  +--+  +  +  +  +
|                  gg |
+  +  +  +  +  +  +  +
|                  CC |
+--+--+--+--+--+--+--+
```

Figure 1: Example show case of a game board with 4×7 cells

¹See https://en.wikipedia.org/wiki/Ricochet_Robots

```

+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+
|##          | |          CC| | ##### CC|
+##+ + +---+---+ + + + + +---+---+ +##+ + + +---+---+ +##+ +
|##  AA|          | |  AA|  ##| |  ##|  ##| |
+##+ +---+ + + + + + + +---+ + + +##+ + + +---+ + +##+ +
|##          gg  | |          gg ##| |          AA  |
+##+ + + + + + + + + + + + +##+ + + + + + + + + + + + +
|#####BB CC| |          BB ##| |          BB  |
+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+

```

Figure 2: Example moves to solve the game configuration from figure 1.

game element. In order to win the game, a robot has to stop in the same cell as the goal, thus the game has not been won if a robot merely slides through the goal cell.

In figure 2 we see a solution to the game from figure 1. The solution consists of six moves:

1. BB -> south
2. BB -> east
3. CC -> north
4. AA -> north
5. AA -> east
6. AA -> south

There are many more solutions to this configuration, many also consisting of six moves — can you find some of them? (I highly doubt there is a solution using under six moves, do not waste your time looking for one!).

2 Implementation guidelines

The following are guidelines for your implementation. These are for your own help and their presence is a guide to your design, hence you are allowed to make modifications and extensions as long as they increase the readability and elegance of your solution.

2.1 User interface

The user interface of your game is the console and as such, the first step in achieving a working game is to implement a way to show the game board including robots, walls and other game elements.

As mentioned above, every board must have outer walls, and in order to also handle inner walls, each cell can have a south facing wall and/or an east facing wall, and nothing else! E.g. in figure 1 we see that the cell with position (2,3) has both a south and east facing wall, while the cell with position (1,4) only has

a south facing wall. Apart from walls, each cell can show a string consisting of two characters.

You are to implement a class called `BoardDisplay` which should be a grid of cells. Each cell should contain some content that can be displayed. With this in mind, a minimum signature of this class can be seen in listing 1.

```
1 public class BoardDisplay {
2     public BoardDisplay(int rows, int cols) {...}
3     public void set(int x, int y, String content, ...) {...}
4     public void setSouthWall(int x, int y) {...}
5     public void setEastWall(int x, int y) {...}
6     public void show() {...}
7 }
```

Listing 1: Minimal methods for the `BoardDisplay` class.

The `set` method should place some content represented as a string at a specific location specified by `x` and `y`, while the `setSouthWall` and `setEastWall` methods should be used to place inner walls. The `show` method should print the entire board to the console.

2.2 Board elements

```
1 public abstract class BoardElement {
2     public abstract void renderOn(BoardDisplay display);
3     public abstract ... interact(Robot robot, Direction direction);
4     public boolean gameOver(List<Robot> robots) {
5         return false;
6     }
7 }
8
9 public class Robot extends BoardElement {
10     private Position position;
11     public Robot(int x, int y, String name) {...}
12     public void step(Direction direction) {...}
13 }
14
15 public class Goal extends BoardElement {
16     public Goal(int x, int y, String name) {...}
17     @Override
18     public boolean gameOver(List<Robot> robots) {...}
19 }
```

Listing 2: Some code outlines for board elements. Note that the return type of the `interact` method is not specified, you should come up with a proper return type.

The next step is to define the pieces — or board elements — we wish to put on our board, and therefore we need to have an abstract `BoardElement` class (why does it make sense for it to be abstract?) that exposes at least the two abstract methods

as seen in listing 2. Furthermore we need robots on our board as well as a goal, both of these are board elements, and thus should inherit from the `BoardElement` class as seen in listing 2.

BoardElement: The `renderOn` method should, given a `BoardDisplay`, render the `BoardElement` on the display, and the `interact` method should, given a robot and a direction², decide the board element's influence on the given robot before it is moved.

Robot: The robot should have a field specifying its position on the board, a constructor parameterized by the starting position of the robot and its name, and a `step` method that moves the robot one step in the given direction.

Goal The goal should also be given a position and name, and it should give its own implementation of the `gameOver` method, which is used for deciding if the game should end or not.

Both the `Robot` and `Goal` class should of course implement the abstract methods inherited from `BoardElement`. Next you need to create three additional concrete classes (all should be a `BoardElement`) in order to handle inner and outer walls:

- `BoardFrame` which is used to represent the outer walls of the board. The class should have a constructor parameterized by the number of rows and columns of the board.
- `VerticalWall` which is used to represent inner vertical walls. The class should have a constructor parameterized by the starting location of the wall — x and y — and how many cells it occupies — n . If n is positive the wall runs from north to south, if n is negative it runs from south to north. The wall is located on the east side of the cells.
- `HorizontalWall` which is used to represent inner horizontal walls. The class should have a constructor parameterized by the starting location of the wall — x and y — and how many cells it occupies — n . If n is positive the wall runs from west to east, if n is negative it runs from east to west. The wall is located on the south side of the cells.

2.3 Game logic

Having almost all the ingredients of a game, we need to put them all together. This should be done in a `Game` class which should contain a `Play` method for starting the game as seen in listing 3.

```
1 public class Game {
2     public Game(...) {...};
3     public void play() {...};
4 }
```

Listing 3: Methods needed for the `Game` class.

²Notice how this implies we have some notion of direction — north, south, east and west

The play method should contain a loop in which all of the game logic is handled. During an iteration of this loop the following should happen: (1) The board should be shown, in addition to how many moves has been used thus far, the names of the available robots, and whatever information you might deem relevant. (2) User input should be fetched, e.g. let the user select a robot by typing its name, afterwards the robot can be controlled by typing movement commands like 'wasd' or 'north', 'south', 'east', 'west'. (3) After a move, check if the game has ended, and if so, show an appropriate end of game display. In order to make the experience more enjoyable you should use the `ClearConsole` method from assignment 3 between each iteration to avoid cluttering the console.

In order to lighten the load on the Game class we will also make a Board class with the methods shown in listing 4.

```
1 public class Board {
2     public Board(...) {...};
3     public void addRobot(Robot robot) {...};
4     public void addElement(BoardElement element) {...};
5     public void move(Robot robot, Direction direction) {...};
6     public List<BoardElement> elements() {...};
7     public List<Robot> robots() {...};
8 }
```

Listing 4: Methods needed for the Board class.

The idea is, that the Game class should *have* a board and make use of it in the game loop.

Note: The Board class should not keep track of *where* the board elements are, they should do this themselves, and Board should not render board elements, but it should *have* a BoardDisplay which the elements can use to render themselves on. Given the signature in listing 4 think about the data structure(s) that should be used for storing the different game elements.

2.4 Game extensions

You should do at least two extensions to the base game described in the previous sections. Add a *short* description of these in a text document that should be included in your repository when handing in. Possible extensions are

- Teleporter: On collision the robot is moved elsewhere (does it keep sliding?).
- Moveable walls: Walls that change position every x turns, or perhaps when a robot interacts with a lever.
- Colorful game elements: Introduce colors to the game elements, see the previous assignment for details on how to use colors in the terminal.
- Bombs: On collision a bomb explodes destroying the robot and possibly walls in the surrounding area.

- Level loading: Make it possible to load a level from file, i.e. the starting configuration of the board, including walls, robots and other game elements is all read from file.

You are also free to use your imagination and come up with your own extensions.

3 Submission

The assignment should be handed in on your GitHub repository in a folder named Assignment4. Be sure to commit and push your solution.

The Moodle Assignment 4 submission is a text file named Assignment4.txt which only contains the URL for your repository.

The assignment should be handed in no later than 23:59 on May 15.